

हेलो! आपका स्वागत है **Unit 17: Fetch API & JSON** की complete study class में.

मैंने आपके workspace में, unit 17 fetch api and json folder के अंदर ये complete study files write कर दी हैं:

- Study Guide File: chapter17_fetch_json.md
- Code Examples File: fetch_json_examples.js
- Practice Exercises File: fetch_json_practice.js

चलिए अब APIs communication flow, JSON serialization mechanisms, HTTP requests triggers, header/body attributes configurations ko simple Hinglish explanation ke sath samajhna shuru karte hain.

PART 1: API & JSON BASICS

Frontend aur Backend server ke beech data sharing isi structural format ke through perform hoti hai.

1. API Kya Hai?

What is it? API (Application Programming Interface) ek intermediate pipeline software tool hota hai jo client web browsers ko database servers se link data communicate.

The Basic Flow: Client (Frontend web page) → Requests parameters → API routing → Server database process → Returns Response.

Real-life Example: Waiter in restaurant: waiter (API) kitchen (server) se food (response) lekar table (client) ko serve karta hai.

2. JSON Format & Conversion APIs

What is it? JSON (JavaScript Object Notation) lightweight text structure data exchange format hai.

- Keys double quotes "" keys hold.
- **JSON.stringify(obj)**: Converts JS Object to JSON String text.
- **JSON.parse(str)**: Parses JSON string text to JS Object.

Simple example

javascript

```
let userObj = { name: "Ravi", score: 100 };
```

```
let jsonStr = JSON.stringify(userObj);  
console.log(jsonStr);  
let parsed = JSON.parse(jsonStr);  
console.log(parsed.name);  
Output {"name":"Ravi","score":100} Ravi
```

PART 2: THE FETCH API (GET & POST)

1. Fetch GET Request

What is it? `fetch()` runs network asynchronous connections to retrieve data. Returns Promise.

Why is `response.json()` needed? Fetch response body returns stream reader stream. `response.json()` converts stream to actual parseable JSON object models.

Simple example (async/await)

javascript

```
async function getData() {  
    let res = await fetch("https://jsonplaceholder.typicode.com/posts/1");  
    let data = await res.json();  
    console.log("Title: " + data.title);  
}  
getData();
```

2. Fetch POST Request

What is it? Sends payload data to save/update resources inside database server.

- **method:** Set to "POST".
- **headers:** Metadata parameters (Content-Type: application/json).
- **body:** Stringified JSON data packet.

Difference: GET vs POST

- GET read values only (no request body).
- POST creates new entries (requires header/body payloads).

Simple example

javascript

```
async function sendData() {  
  let payload = { title: "New Item" };  
  
  let res = await fetch("https://jsonplaceholder.typicode.com/posts", {  
    method: "POST",  
    headers: {  
      "Content-Type": "application/json"  
    },  
    body: JSON.stringify(payload)  
  });  
  
  let data = await res.json();  
  
  console.log("Success. ID:", data.id);  
}  
  
sendData();
```

Output Success. ID: 101

PART 3: ADVANCED EXCEPTIONS & LOADINGS

1. HTTP Status & response.ok

What is it?

- **response.status:** Returns numeric status codes (e.g. 200 success, 404 not found, 500 server crash).
- **response.ok:** Boolean checker status. Returns true if status range is 200-299.

2. Safe Fetch Error Handling

IMPORTANT POINT: fetch() does not automatically reject if server returns 404 / 500. It only rejects on total network DNS drops. We must check response.ok manually and throw exceptions.

Proper Error handling Example:

javascript

```
async function fetchSafe() {  
  try {  
    let res = await fetch("https://jsonplaceholder.typicode.com/invalid-url");  
    if (!res.ok) {  
      throw new Error(`HTTP Error status: ${res.status}`);  
    }  
    let data = await res.json();  
    console.log(data);  
  } catch(err) {  
    console.log("Caught: " + err.message);  
  }  
}  
  
fetchSafe();
```

Output Caught: HTTP Error status: 404

3. Loading States

What is it? Track loader spinner visual flags when API fetching network requests are pending.

Simple logic:

javascript

```
let loading = false;  
  
async function fetchWithLoading() {  
  loading = true;  
  console.log("LOADING...");  
  try {  
    await fetch("https://jsonplaceholder.typicode.com/posts/1");  
    console.log("SUCCESS!");  
  } catch(e) {
```

```
    console.log("ERROR!");
  } finally {
    loading = false;
    console.log("FINISHED");
  }
}

fetchWithLoading();
```

QUICK REVISION SUMMARY

- **JSON Conversion:** `stringify()` serialize object to JSON string; `parse()` deserialize JSON string back to Object.
 - **GET vs POST:** GET gets data from server; POST sends body payload to server to create resources.
 - **Fetch logic:** Returns Promise. Response stream requires asynchronous `.json()` parsing.
 - **Error handling:** `fetch()` only rejects on network loss. Status errors (404/500) must be validated using `response.ok` manually.
 - **Loading states:** loading spinner tracks request lifecycle (true before call, false inside `finally`).
-

IMPORTANT DEFINITIONS (INTERVIEW-READY)

1. **JSON Serialization:** converting live memory data object trees into textual JSON strings using `JSON.stringify()`.
 2. **response.ok:** Boolean check property of Fetch Response that is true if HTTP code lies in 200-299 success range.
 3. **HTTP Status Code:** Number code returns indicating request completion status categories (2xx success, 4xx client errors, 5xx server errors).
 4. **Header Content-Type:** Metadata parameter informing server of the incoming request body data format.
 5. **API Endpoint:** Designated URL gateway where applications query resources.
-

IMPORTANT INTERVIEW QUESTIONS & ANSWERS

Q1. Why doesn't `fetch()` reject on 404 or 500 status codes?

Ans: `fetch()` is designed to check if the network request completed successfully. An HTTP response from the server (even a 404 or 500 error) means the network call finished, so the Promise resolves. It only rejects if the request is blocked by a network outage or invalid DNS.

Q2. Difference between GET and POST methods?

Ans: GET retrieves data from the server and does not accept a request body. POST sends new data payload inside the request body to create resources on the server.

Q3. Why is `response.json()` a Promise?

Ans: Reading the body of a raw HTTP response is a stream-based asynchronous process. JavaScript returns a Promise that resolves once the entire stream has been processed and parsed as JSON.

Q4. Difference between `JSON.stringify()` and `JSON.parse()`?

Ans: `JSON.stringify()` serializes a JavaScript object into a JSON string text. `JSON.parse()` deserializes a JSON string back into a JavaScript object.

Q5. What is the use of `Content-Type: application/json` header?

Ans: It tells the backend server that the request body payload data is formatted as JSON text, so the server can parse it correctly.

Q6. Predict output:

```
javascript
let json = '{"name":"Aman"}';
let obj = JSON.parse(json);
console.log(typeof obj);
```

Ans: "object".

Q7. How do you handle network failure vs server error in `fetch()`?

Ans: Network failures are caught by `.catch()` or try-catch blocks automatically. Server errors (like 404/500) must be caught manually by evaluating `if (!response.ok)` and throwing a custom error.

Q8. Predict output:

```
javascript
let user = { id: 1 };
```

```
console.log(typeof JSON.stringify(user));
```

Ans: "string".

Q9. What does status code 201 mean?

Ans: Created. (Successful API POST request response).

Q10. What does status code 401 mean?

Ans: Unauthorized. (Access credentials missing or invalid).

Q11. Predict output:

```
javascript  
  
let text = "{name: 'Rahul'}";  
  
try {  
    JSON.parse(text);  
} catch (e) {  
    console.log(e.name);  
}
```

Ans: "SyntaxError". (JSON format keys and values must be enclosed in double quotes "").

Q12. What does response.ok return if status code is 304 (Not Modified)?

Ans: false. (Success is strictly defined in range 200-299).

Q13. Can you send files inside a JSON request body?

Ans: No. JSON only supports textual values. Files are commonly sent using Multipart FormData formats.

Q14. Predict output:

```
javascript  
  
fetch("https://invalid-host-test.com")  
    .catch(err => console.log("A"));
```

Ans: "A". (Domain lookup DNS failure triggers promise rejection).

Q15. Why do we call event.preventDefault() inside form submit handlers before fetching?

Ans: To stop the default page reload, allowing us to send the fetch request asynchronously in the background.

Q16. Predict output:

javascript

```
let obj = { x: undefined };  
console.log(JSON.stringify(obj));
```

Ans: "{}". (Undefined properties are omitted during JSON stringify serialization).

Q17. What is default HTTP method inside fetch() if none is specified?

Ans: GET.

Q18. Predict output:

javascript

```
console.log(JSON.parse("true"));
```

Ans: true (boolean true). (Valid JSON values are parsed to their corresponding JS primitive types).

Q19. What does status code 403 represent?

Ans: Forbidden. (Authenticated client lacks permissions for the resource).

Q20. Can we run a request body payload inside a GET request?

Ans: Standard specifications forbid request bodies in GET requests. If sent, most servers ignore it or reject the request.



PRACTICE QUESTIONS

Question 1: Fetch and Log Title

- **Question:** Fetch data from API <https://jsonplaceholder.typicode.com/posts/2> and log the title.
- **Solution:**

javascript

```
async function fetchPostTitle() {  
  try {  
    let res = await fetch("https://jsonplaceholder.typicode.com/posts/2");  
    if (!res.ok) throw new Error("HTTP error: " + res.status);  
    let data = await res.json();  
    console.log("Title: " + data.title);  
  }  
}
```



```

    } catch (err) {

        console.log("Fetch failed: " + err.message);

    }

}

fetchPostTitle();

```

- **Explanation:** Performs async GET, checks res.ok, parses to JSON, and logs the title field.

Question 2: POST request dispatch validation

- **Question:** Send post request to server containing user properties: { name: "Ravi", job: "QA" }.
- **Solution:**

```

javascript

async function registerUser() {

    let payload = { name: "Ravi", job: "QA" };

    try {

        let res = await fetch("https://reqres.in/api/users", {

            method: "POST",

            headers: { "Content-Type": "application/json" },

            body: JSON.stringify(payload)

        });

        let outcome = await res.json();

        console.log("Registered ID:", outcome.id);

    } catch (e) {

        console.log("Error:", e.message);

    }

}

registerUser();

```

- **Explanation:** Sends stringified body with JSON content-type header using POST method.

Question 3: Response status checks

- **Question:** Write a function checking if URL matches status code 404.
- **Solution:**

javascript

```
async function testNotFound() {  
  try {  
    let res = await fetch("https://jsonplaceholder.typicode.com/invalidpage");  
    console.log("Status:", res.status); // 404  
    console.log("Is OK?", res.ok); // false  
  } catch (e) {  
    console.log("Error: " + e.message);  
  }  
}  
  
testNotFound();
```

- **Explanation:** Resolves URL and checks status properties.

10:10 PM